COMMONWEALTH OF AUSTRALIA
Copyright Regulations 1969
This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice

Prepared by: [Arun Konagurthu]
Extended by: Taylor Kearney

FIT3155: Advanced Algorithms and Data Structures

Week 1: **Gusfield's Z-Algorithm and Exact Pattern Matching**

Faculty of Information Technology, Monash University

What is covered here?

Linear-time approaches to the exact pattern matching problem on strings

- This week
  - Gusfield's Z-algorithm
- Next week
  - Boyer-Moore's algorithm
  - Knuth-Morris-Pratt's algorithm

Source material and recommended reading

- Dan Gusfield, Algorithms on Strings, Trees and Sequences, Cambridge University Press. (Chapters 1-2).

Exact pattern matching: Introduction

The exact pattern matching problem

Given a reference text **txt**$[1 \ldots n]$ and a pattern **pat**$[1 \ldots m]$, find **ALL** occurrences, if any, of **pat** in **txt**.

Example

If, **txt**$[1 \ldots 12] =$ b b a b a x a b a b a y (positions 1 2 3 4 5 6 7 8 9 10 11 12) and **pat**$[1 \ldots 3] =$ a b a (positions 1 2 3) then **pat** occurs in **txt** at positions 3,7 and 9.

	1	2	3	4	5	6	7	8	9	10	11	12
txt:	b	b	a	b	a	x	a	b	a	b	a	y
pat:			✓ a 1	✓ b 2	✓ a 3							
pat:							✓ a 1	✓ b 2	✓ a 3			
pat:									✓ a 1	✓ b 2	✓ a 3	

Exact pattern matching: Introduction

- Problem arises in innumerable applications
  - Word processing – `grep` command in Unix
  - Search Engines – Google
  - Library catalogs
  - ⋮

Naïve algorithm

```
for i from 1 to n-m+1:  // for each position i in txt
    // check if txt[i..i+m-1]==pat[1..m]
    for j from 1 to m:
         if txt[i+j-1] != pat[j] then
             break
         endif
    endfor
    //if pat matched fully at i, print position in txt
    if (j == m+1) print i
endfor
```

Naïve algorithm

	1	2	3	4	5	6	7	8	9	10
txt:	a	a	a	a	a	a	a	a	a	a
iteration-1	✓	✓	✓							
pat:	a	a	a							
	1	2	3							
iteration-2		✓	✓	✓						
pat:		a	a	a						
		1	2	3						
iteration-3			✓	✓	✓					
pat:			a	a	a					
			1	2	3					
⋮					⋮					
iteration-8								✓	✓	✓
pat:								a	a	a
								1	2	3

How many comparison of symbols does this approach perform in worst case?

Total number of comparisons, in the **worst case**, is $(n - m + 1) \times m = \mathcal{O}(mn)$.

In general, naïve approach involves too many unnecessary comparisons

	1	2	3	4	5	6	7	8	9	10	11	12	13
txt:	x	a	b	x	y	a	b	x	y	a	b	x	z
iteration-1	✗												
pat:	a	b	x	y	a	b	x	z					
	1	2	3	4	5	6	7	8					
iteration-2		✓	✓	✓	✓	✓	✓	✓	✗				
pat:		a	b	x	y	a	b	x	z				
		1	2	3	4	5	6	7	8				
iteration-3			✗										
pat:			a	b	x	y	a	b	x	z			
			1	2	3	4	5	6	7	8			
iteration-4				✗									
pat:				a	b	x	y	a	b	x	z		
				1	2	3	4	5	6	7	8		
iteration-5					✗								
pat:					a	b	x	y	a	b	x	z	
					1	2	3	4	5	6	7	8	
iteration-6						✓	✓	✓	✓	✓	✓	✓	✓
pat:						a	b	x	y	a	b	x	z
						1	2	3	4	5	6	7	8

Overall 20 comparisons in the naïve approach, on this example.

Ideas for speeding up the naïve method

Shift more purposefully: Shift **pat** by > 1 position after each iteration, whenever possible.

Shift always safely: Never shift so far as to miss any exact occurrence of **pat** in **txt**.

A smarter approach can reduce unnecessary comparisons

Scenario 1:

A smarter approach can gather that, after the mismatch in iteration-2 (for example), there is enough information to shift **pat** by > 1 position character in the next iteration.

	1	2	3	4	5	6	7	8	9	10	11	12	13
txt:	x	a	b	x	y	a	b	x	y	a	b	x	z
iteration-1	✗												
pat:	a	b	x	y	a	b	x	z					
	1	2	3	4	5	6	7	8					
iteration-2		✓	✓	✓	✓	✓	✓	✓	✗				
pat:		a	b	x	y	a	b	x	z				
		1	2	3	4	5	6	7	8				
iteration-3						✓	✓	✓	✓	✓	✓	✓	✓
pat:						a	b	x	y	a	b	x	z
						1	2	3	4	5	6	7	8

How does this approach achieve this?

By the end of iteration-2, the approach has learnt that **pat**$[1 \ldots 7] =$ **txt**$[2 \ldots 8]$. It can gather that **pat**$[1] =$ 'a' does not occur until position 6 in **txt**. This is enough to conclude there can be no exact matches of **pat** starting positions 3 to 6 in **txt**, and hence can shift it by $4 > 1$ positions **safely**.

Overall, this 'smarter' algorithm saves 3 comparisons on this example.

An even smarter approach can reduce unnecessary comparisons even more

Scenario 2:

An 'even smarter' algorithm can gather more information, beyond scenario 1, and render more comparisons redundant.

	1	2	3	4	5	6	7	8	9	10	11	12	13
txt:	x	a	b	x	y	a	b	x	y	a	b	x	z
iteration-1	✗												
pat:	a	b	x	y	a	b	x	z					
	1	2	3	4	5	6	7	8					
iteration-2		✓	✓	✓	✓	✓	✓	✓	✗				
pat:		a	b	x	y	a	b	x	z				
		1	2	3	4	5	6	7	8				
iteration-3									✓	✓	✓	✓	✓
pat:						a	b	x	y	a	b	x	z
						1	2	3	4	5	6	7	8

How does this algorithm achieve this?

This algorithm preprocesses **pat** and learns that **pat**$[1...3]$ ('abx') is same as **pat**$[5...7]$. After iteration 2, it knows **pat**$[5...7]$ = **txt**$[6...8]$, allowing it to shift such that **pat**$[1...3]$ aligns under **txt**$[6...8]$ without re-comparing those characters in iteration-3, saving 3 more comparisons.

Overall, this 'even smarter' algorithm saves 6 comparisons over 'naïve'.

Take home message from these illustrated examples

- The examples shown in the previous slides illustrate the kind of ideas that allow unnecessary comparisons to be skipped/jumped over.
  - Note, we haven't yet rationalized how an algorithm can **efficiently** implement such ideas shown in previous scenarios.
- Over weeks 1 and 2, we will study 3 remarkable algorithms that can be implemented to run pattern matching in **linear time**.
  - **Gusfield's Z-Algorithm**
  - **Boyer-Moore's Algorithm**
  - **Knuth-Morris-Pratt's Algorithm**

1. Gusfield's Z-Algorithm

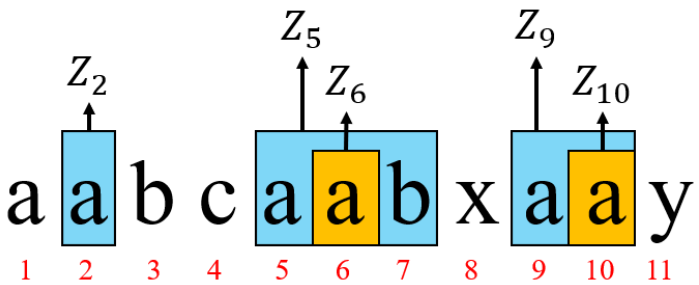
Comments:

- This is actually a linear-time algorithm to **preprocess** a given string.
- After preprocessing, the output from this algorithm (Z_i-values) can be used to address a **versatile set of problems** that arise in strings, including **linear-time pattern matching**.
- In this unit you will witness its various uses.

Gusfield's Z-algorithm – Defining Z_i

Definition of Z_i:

For a string **str**$[1 \ldots n]$, define Z_i (for each position $i > 1$ in **str**) as the **length** of the **longest substring** starting at **position** i of **str** that **matches its prefix** (i.e., **str**$[i \ldots i{+}Z_i\text{-}1] =$ **str**$[1 \ldots Z_i]$).



Example

```
      1 2 3 4 5 6 7 8 9 10 11
str= a a b c a a b x a a  y
```

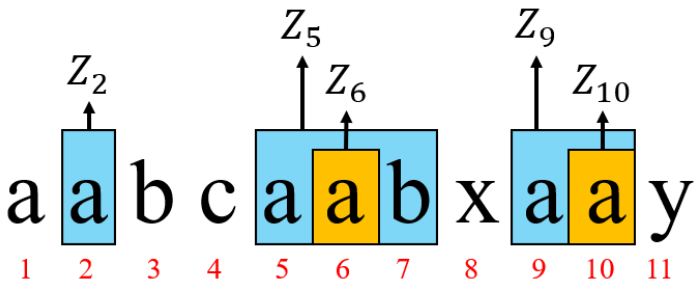
$Z_1 =$ trivially always the length of the string. So, ignore this.

$Z_2 = 1$	$Z_7 = 0$
$Z_3 = 0$	$Z_8 = 0$
$Z_4 = 0$	$Z_9 = 2$
$Z_5 = 3$	$Z_{10} = 1$
$Z_6 = 1$	$Z_{11} = 0$

Gusfield's Z-algorithm – Defining Z_i-box

Definition of Z-box:

For a string **str**$[1..n]$, **and for any** $i > 1$ **such that** $Z_i > 0$, a Z_i-box is defined as the interval $[i \ldots i{+}Z_i\text{-}1]$ of **str**.



Example

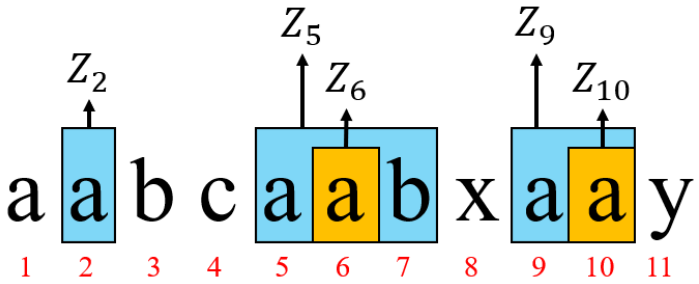
	1	2	3	4	5	6	7	8	9	10	11
str=	a	a	b	c	a	a	b	x	a	a	y

$Z_2 = 1$	$\Longrightarrow$	Z_2-box $= [2..2]$	$Z_7 = 0$	$\Longrightarrow$	Z_7-box = undefined
$Z_3 = 0$	$\Longrightarrow$	Z_3-box = undefined	$Z_8 = 0$	$\Longrightarrow$	Z_8-box = undefined
$Z_4 = 0$	$\Longrightarrow$	Z_4-box = undefined	$Z_9 = 2$	$\Longrightarrow$	Z_9-box $= [9..10]$
$Z_5 = 3$	$\Longrightarrow$	Z_5-box $= [5..7]$	$Z_{10} = 1$	$\Longrightarrow$	Z_{10}-box $= [10..10]$
$Z_6 = 1$	$\Longrightarrow$	Z_6-box $= [6..6]$	$Z_{11} = 0$	$\Longrightarrow$	Z_{11}-box = undefined

Gusfield's Z-algorithm – Defining r_i

Definition of r_i:

For a string **str**$[1..n]$, and for all $i > 1$, r_i is the **right-most endpoint** of all Z-boxes that begin at or before position i.
Alternately, r_i is the largest value of $j+Z_j$-1 over all $1 < j \leq i$, such that $Z_j > 0$.



Example

	1	2	3	4	5	6	7	8	9	10	11
str=	a	a	b	c	a	a	b	x	a	a	y

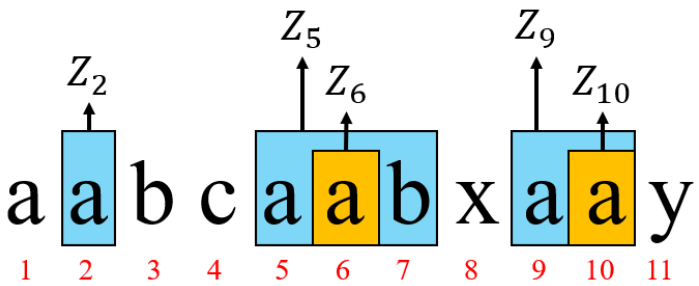
$Z_2 = 1$	$Z_7 = 0$	Z_2-box = [2..2]	$r_2 = 2$	$r_7 = 7$
$Z_3 = 0$	$Z_8 = 0$	Z_5-box = [5..7]	$r_3 = 2$	$r_8 = 7$
$Z_4 = 0$	$Z_9 = 2$	Z_6-box = [6..6]	$r_4 = 2$	$r_9 = 10$
$Z_5 = 3$	$Z_{10} = 1$	Z_9-box = [9..10]	$r_5 = 7$	$r_{10} = 10$
$Z_6 = 1$	$Z_{11} = 0$	Z_{10}-box = [10..10]	$r_6 = 7$	$r_{11} = 10$

Gusfield's Z-algorithm – Defining l_i

Definition of l_i:

For a string **str**$[1..n]$, and for all $i > 1$, l_i is the **left end** of the Z-box that ends at r_i.

In case there is more than one Z-box ending at r_i, then l_i can be chosen to be the left end of **any** of those Z-boxes.

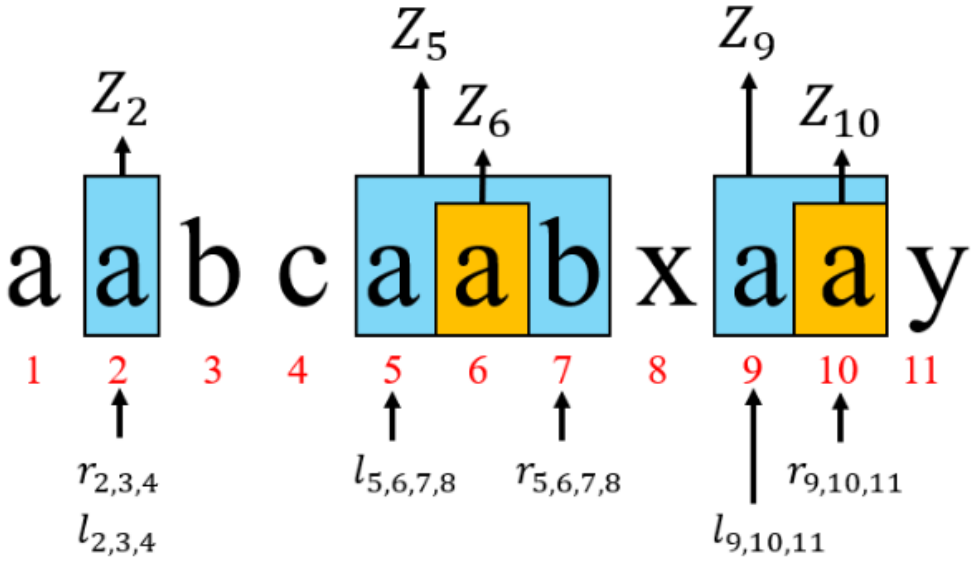


Example

	1	2	3	4	5	6	7	8	9	10	11
str=	a	a	b	c	a	a	b	x	a	a	y

$Z_2 = 1$	$Z_7 = 0$	Z_2-box = [2..2]	$l_2 = 2$	$l_7 = 5$
$Z_3 = 0$	$Z_8 = 0$	Z_5-box = [5..7]	$l_3 = 2$	$l_8 = 5$
$Z_4 = 0$	$Z_9 = 2$	Z_6-box = [6..6]	$l_4 = 2$	$l_9 = 9$
$Z_5 = 3$	$Z_{10} = 1$	Z_9-box = [9..10]	$l_5 = 5$	$l_{10} = 9$
$Z_6 = 1$	$Z_{11} = 0$	Z_{10}-box = [10..10]	$l_6 = 5$	$l_{11} = 9$

Gusfield's Z-algorithm – Example (all things in one slide)



Example

$Z_2 = 1$	Z_2-box $= [2..2]$	$r_2 = 2$	$r_7 = 7$	$l_2 = 2$	$l_7 = 5$
$Z_5 = 3$	Z_5-box $= [5..7]$	$r_3 = 2$	$r_8 = 7$	$l_3 = 2$	$l_8 = 5$
$Z_6 = 1$	Z_6-box $= [6..6]$	$r_4 = 2$	$r_9 = 10$	$l_4 = 2$	$l_9 = 9$
$Z_9 = 2$	Z_9-box $= [9..10]$	$r_5 = 7$	$r_{10} = 10$	$l_5 = 5$	$l_{10} = 9$
$Z_{10} = 1$	Z_{10}-box $= [10..10]$	$r_6 = 7$	$r_{11} = 10$	$l_6 = 5$	$l_{11} = 9$

Main point of Gusfield's Z-algorithm!

- In the previous slides, for any given string, we have defined: $\{Z_i$, Z_i-box, l_i, $r_i\}$.
- Naïve computation of these values, given some string $\texttt{str}[1..n]$ would require $O(n^2)$-time.
- Gusfield's Z-algorithm allows computation of these values, in $O(n)$-time.

Overview of Gusfiled's Z-algorithm

- We compute $Z_i$values **for each successive position** $i \geq 2$ (for 1-based indexing, used in the slides).
- All successively computed array of Z_i values are remembered.
  - Note: Each Z_i-box interval can be computed from its corresponding Z_i value in $O(1)$ time. No need to store the interval information.
- At each iteration, to compute (l_i,r_i) values, you will only needs values of (l_{i-1}, r_{i-1}).
  - Note: no earlier (l_j, r_j) values for $j < i - 1$ are needed ...
  - ...so, temporary variables (l, r) can be used to keep track of the most recently computed (l_{i-1}, r_{i-1}) values to update (l_i,r_i).

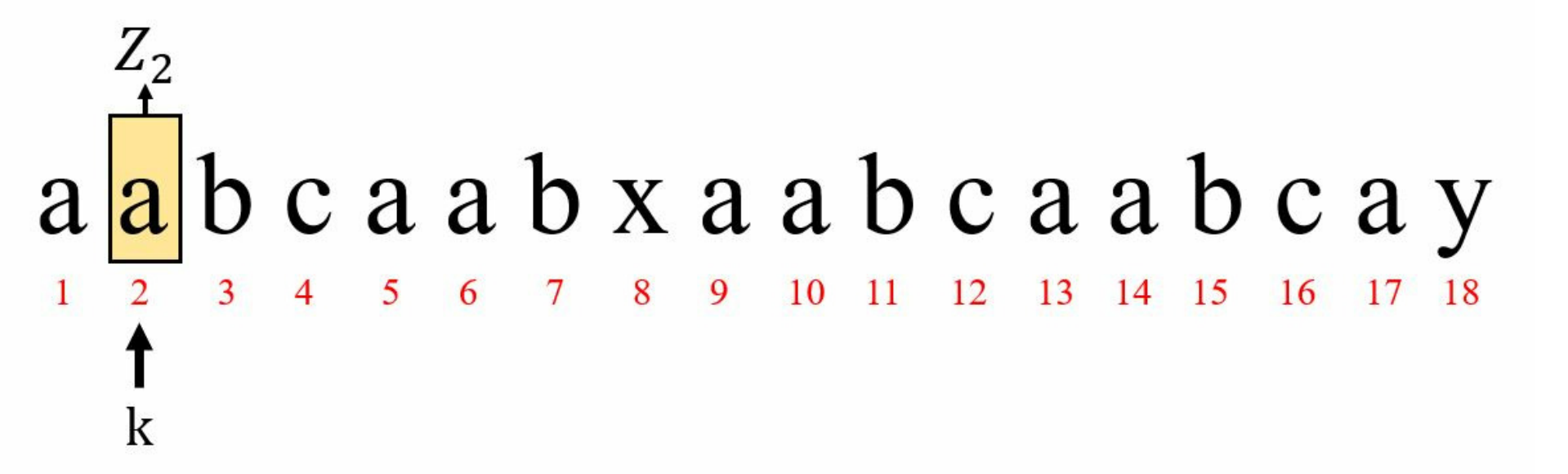
Let's see how this all works out in practice.

(Remember the slides employ 1-based array indexing. In your program, you will have to deal with this in 0-based indexing.)

Z-algorithm – base case

- To begin, compute Z_2 by explicit **left-to-right** comparison of characters **str**$[2 \ldots]$ with **str**$[1 \ldots]$ until a mismatch is found.
- If $Z_2 > 0$
  - set r (i.e., r_2) to $2 + Z_2 - 1 = Z_2 + 1$
  - set l (i.e., l_2) to 2
- else (i.e., if $Z_2 == 0$)
  - set r (i.e., r_2) to 0
  - set l (i.e., l_2) to 0

Z-algorithm – base case example (note: using a new example string from here on)



$Z_2 = 1$ so set $l = 2$ and $r = Z_2 + 1 = 2$.

Z-algorithm – inductive assumption and general cases

Assume inductively . . .

- . . . we have correctly computed the values Z_2 through to Z_{k-1}.
- . . . that r currently holds r_{k-1},
- . . . that l currently holds l_{k-1}.

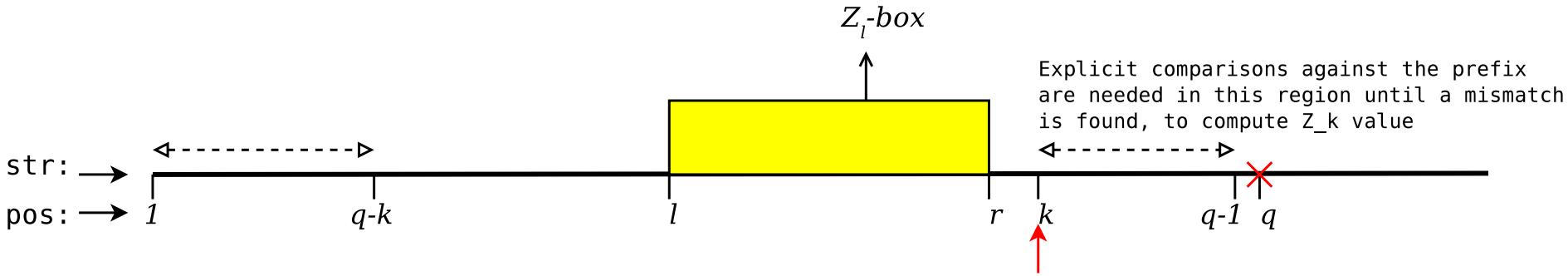
For computing Z_k at position k, these two scenarios arise:

Case 1: If $k > r$

Case 2: Else $k \leq r$

Let's see how to have to handle these two cases.

preprocessing in practice – Case 1: $k > r$



- CASE 1, if $k > r$:
  - Compute Z_k by explicitly comparing characters **str**$[k \ldots q-1]$ with **str**$[1 \ldots q-k]$ until mismatch is found at some $q \geq k$.
  - If $Z_k > 0$:
    - ★ set r (i.e., r_k) to $q-1$.
    - ★ set l (i.e., l_k) to k.
  - Otherwise (i.e. $Z_k = 0$) they retain the same (l, r) values as before

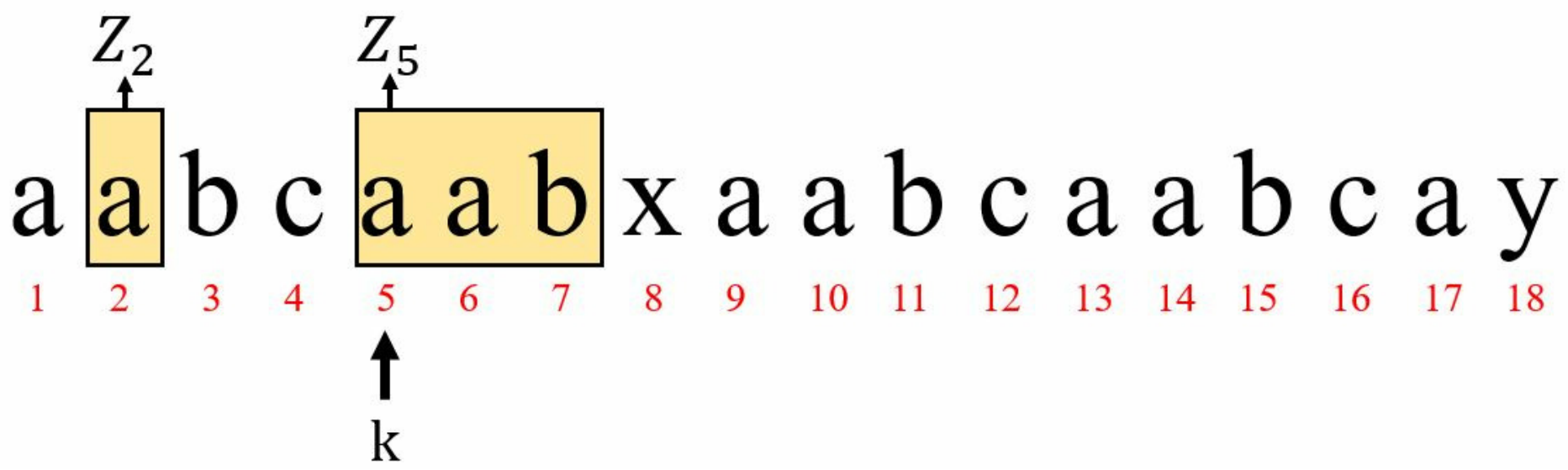
Z-algorithm – Case 1: $k > r$

$Z_3 = 0$ so l and r remain unchanged ($l = 2$ and $r = 2$).

Z-algorithm – Case 1: $k > r$

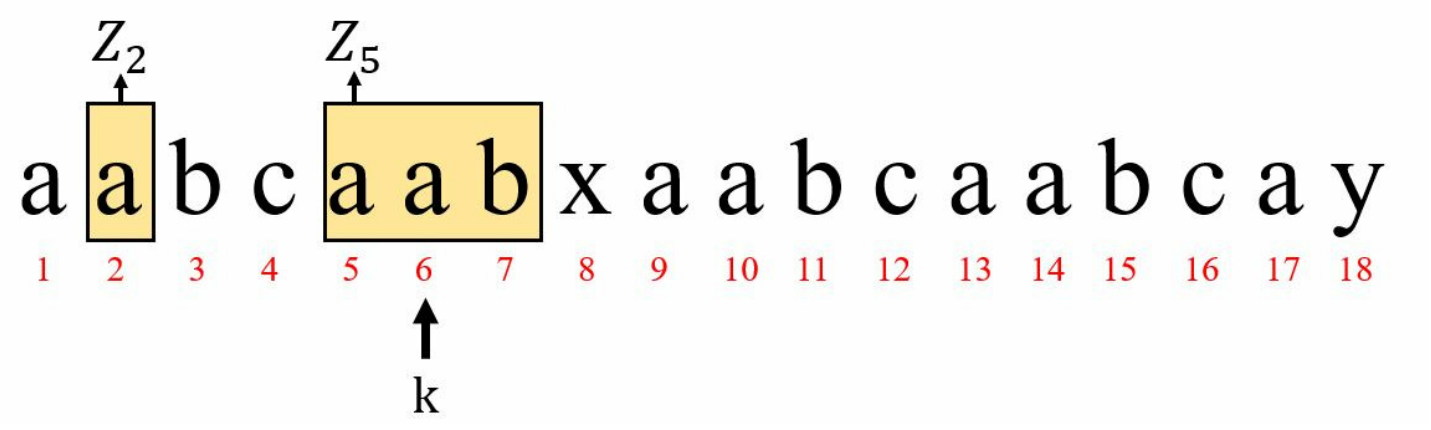
$Z_4 = 0$ so l and r remain unchanged ($l = 2$ and $r = 2$).

Z-algorithm – Case 1: $k > r$



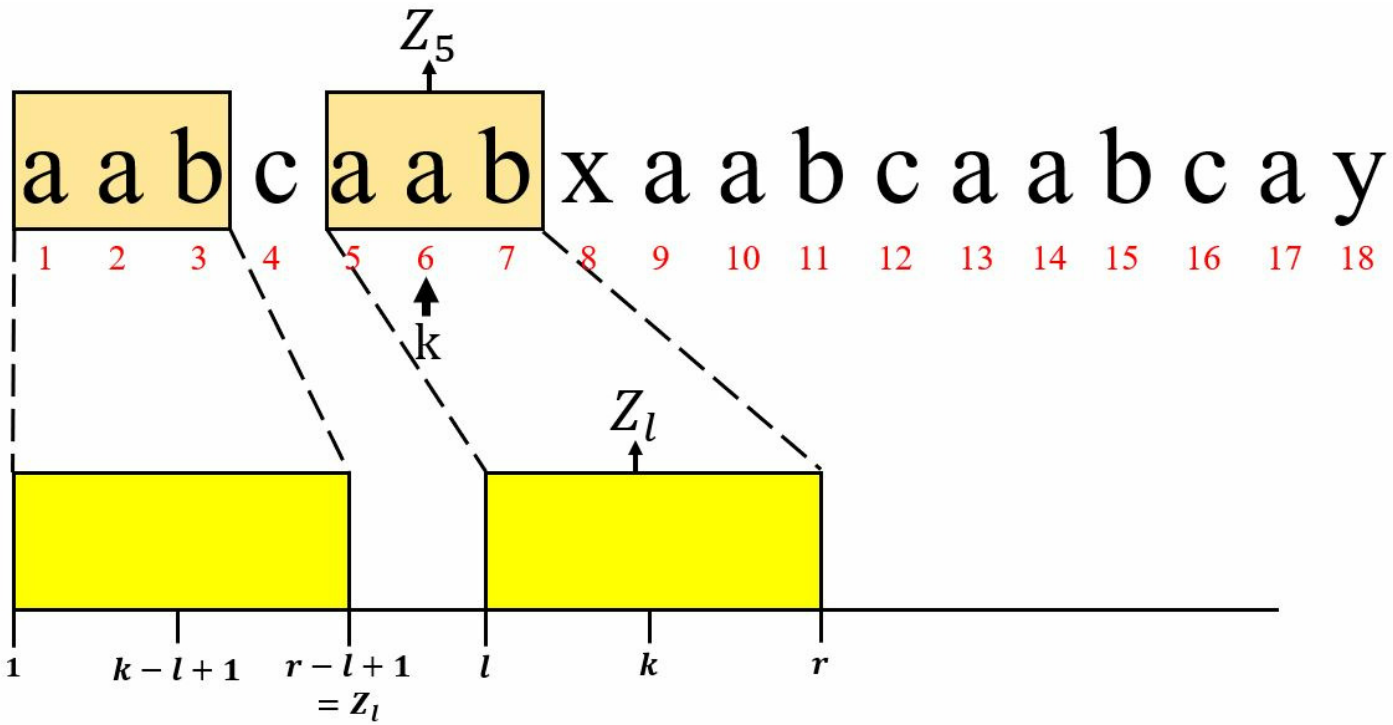
$Z_5 = 3$ so l and r are updated to: $l = 5$ and $r = 7$.

Z-algorithm – Case 2: $k \leq r$



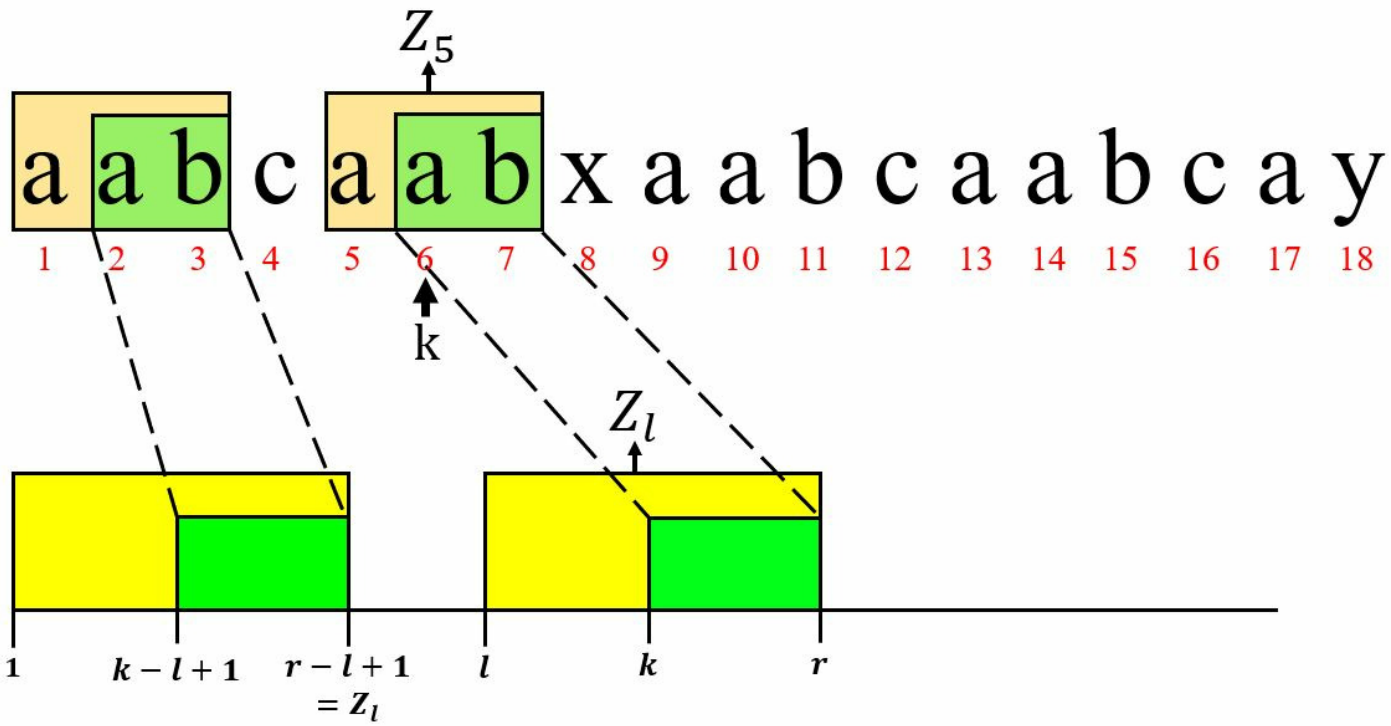
- CASE 2, if $k \leq r$:
  - The character **str**$[k]$ lies anywhere within the substring **str**$[l \ldots r]$ (i.e., anywhere within the Z_l-box).

Z-algorithm – Case 2: $k \leq r$



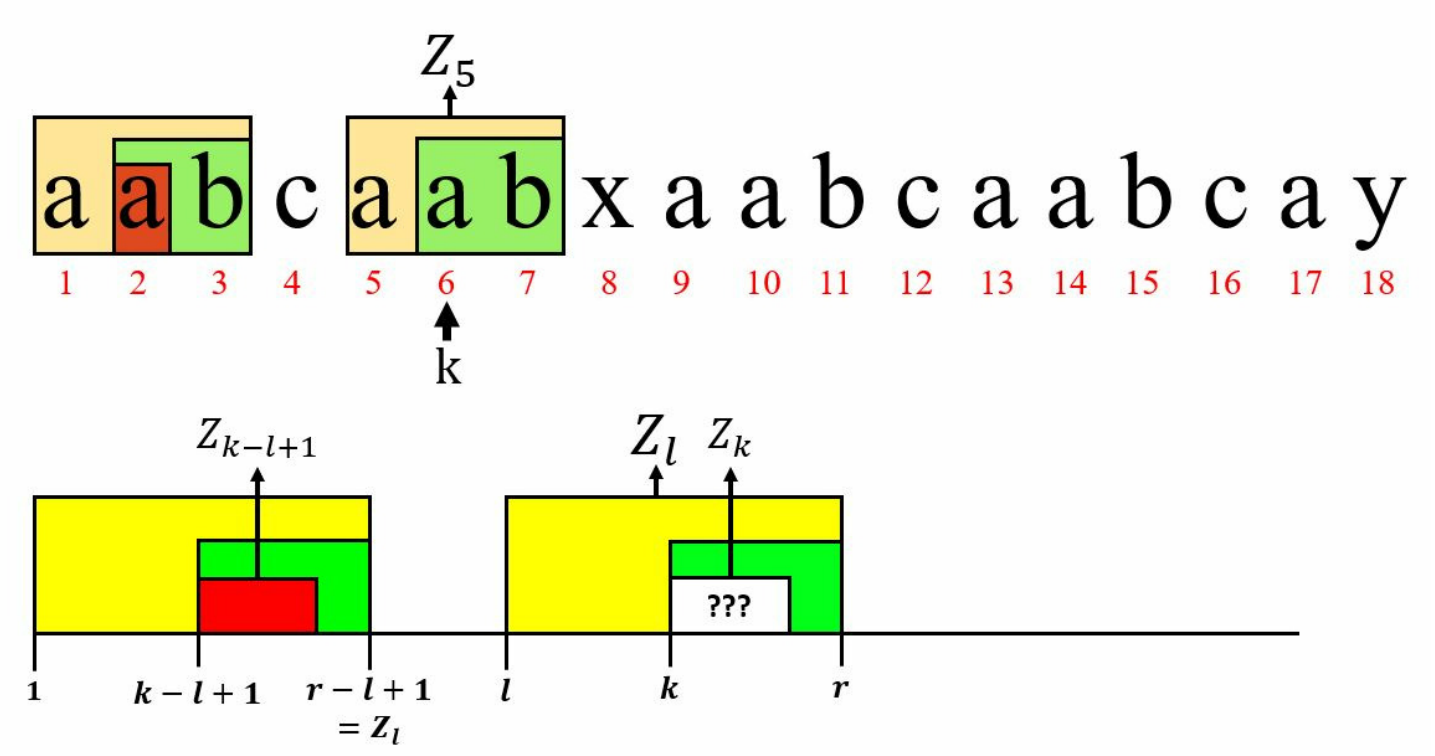
- CASE 2, if $k \leq r$:
  - The character **str**$[k]$ lies anywhere within the substring **str**$[l \ldots r]$ (i.e., anywhere within the Z_l-box).
  - By the definition of Z_l-box, **str**$[l \ldots r]$=**str**$[1 \ldots Z_l]$.

Z-algorithm – Case 2: $k \leq r$



- CASE 2, if $k \leq r$:
  - The character **str**$[k]$ lies anywhere within the substring **str**$[l \ldots r]$ (i.e., anywhere within the Z_l-box).
  - By the definition of Z_l-box, **str**$[l \ldots r]$=**str**$[1 \ldots Z_l]$.
  - This implies the character **str**$[k]$ is identical to **str**$[k-l+1]$
  - By extending this logic, it also implies that the substring **str**$[k \ldots r]$ is identical to **str**$[k-l+1 \ldots Z_l]$.

Z-algorithm – Case 2: $k \leq r$



Question: Can the previously computed value of Z_{k-l+1} (see red box) inform the computation of Z_k (see white box)?

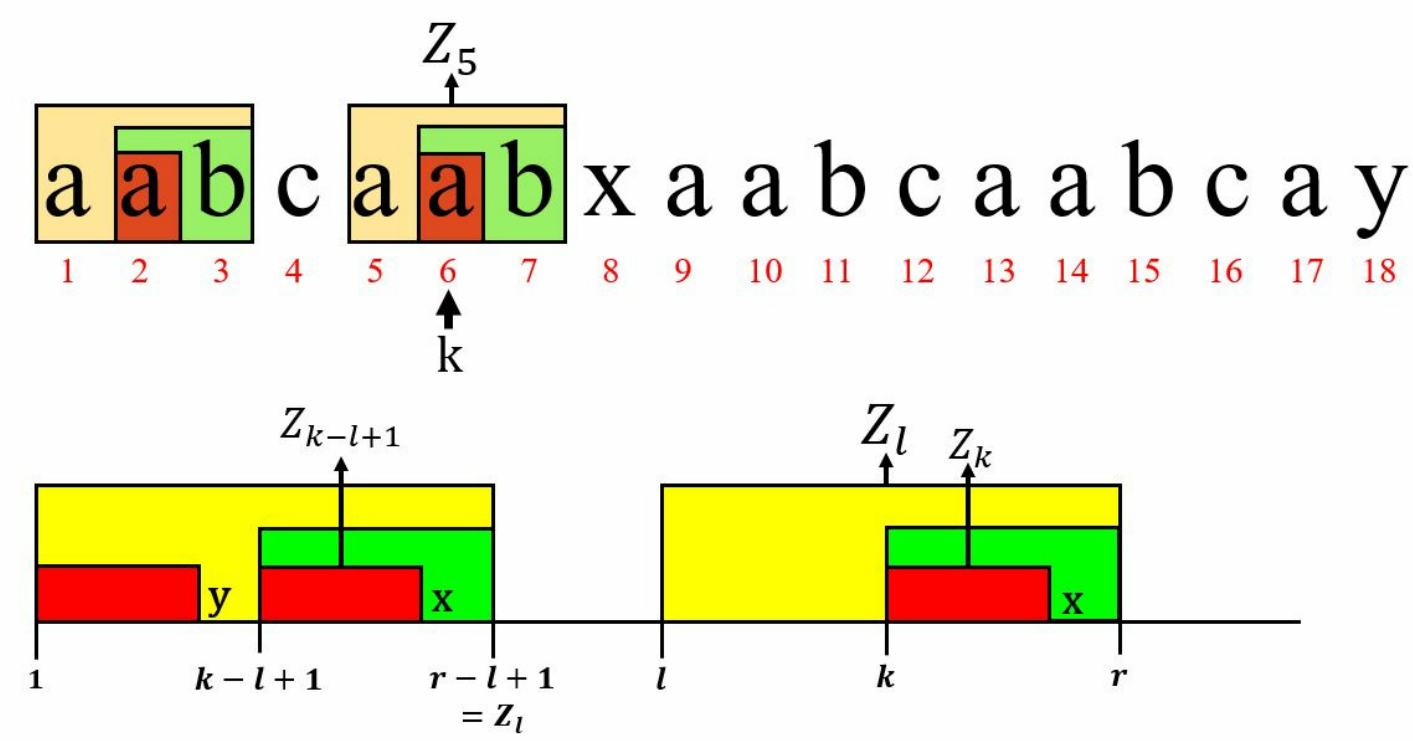
Z-algorithm – overview of handling case 2

In the previous slide, we asked "*can the value of Z_{k-l+1} inform the computation of Z_k?*". The answer is **yes**, and this can be handled by two **sub**-cases: CASES 2a and 2b.

- CASE 2a, if $Z_{k-l+1} < r - k + 1$:
- CASE 2b, else (if) $Z_{k-l+1} \geq r - k + 1$:

Z-algorithm – case 2a mechanics

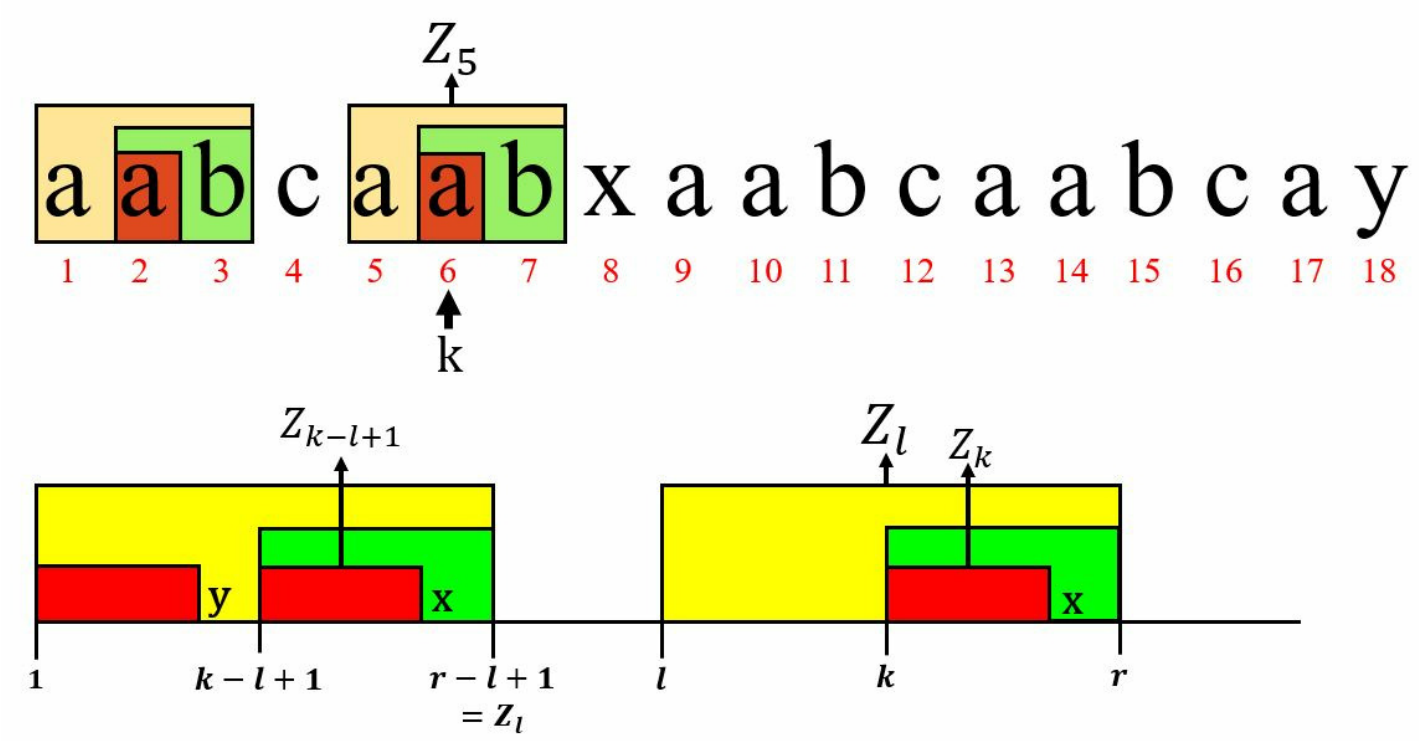
- CASE 2a, if $Z_{k-l+1} < r - k + 1$:



- Therefore, when $Z_{k-l+1} < r - k + 1$
  - set Z_k to Z_{k-l+1}, and
  - r and l values remain unchanged.

Z-algorithm – case 2a mechanics

- CASE 2a, if $Z_{k-l+1} < r - k + 1$:

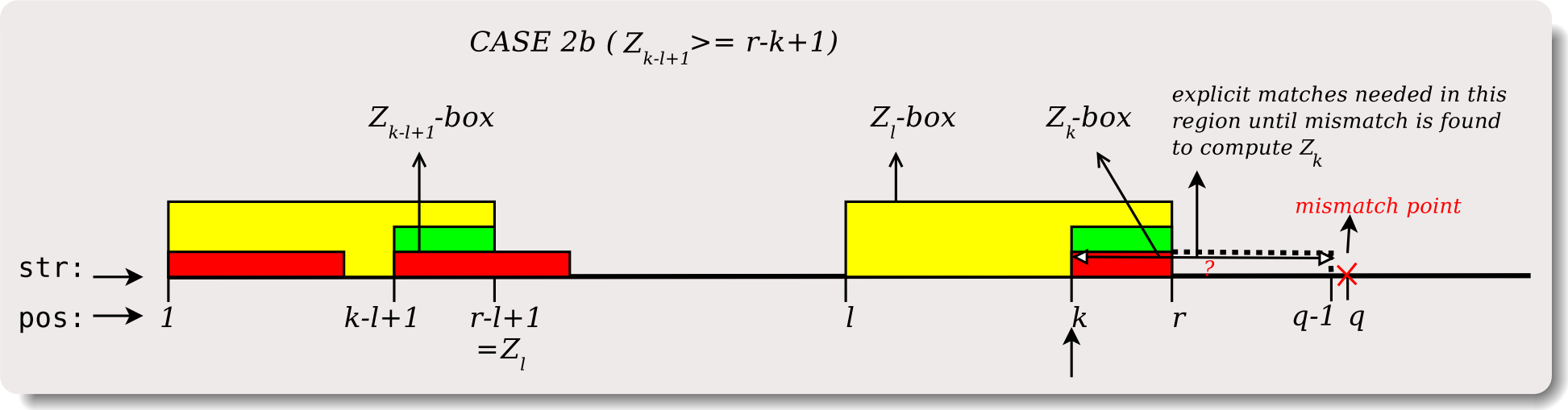


- Therefore, when $Z_{k-l+1} < r - k + 1$
  - set Z_k to Z_{k-l+1}, and
  - r and l values remain unchanged.

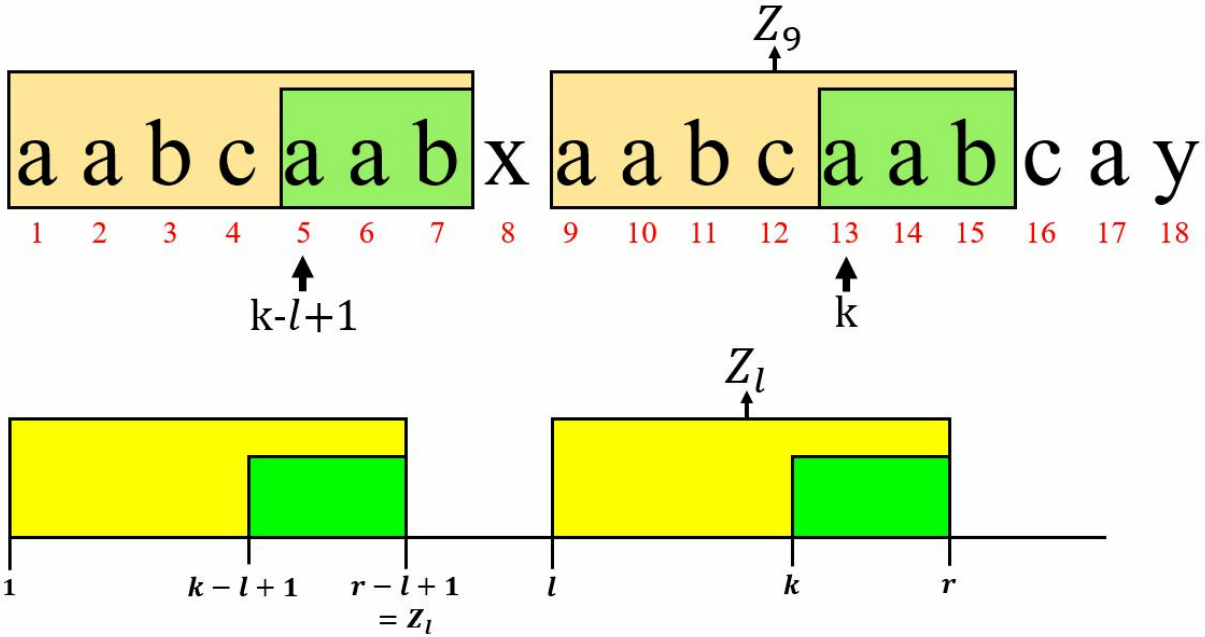
CASE 2a is now complete. Let's now focus on the remaining case 2b.

Z-algorithm – Case 2b

- CASE 2b, if $Z_{k-l+1} \geq r - k + 1$:
  - Z_k must also be $\geq r - k + 1$
  - So, start explicitly comparing **str**$[r+1]$ with **str**$[r-k+2]$ and so on until mismatch occurs.
  - Say the mismatch occurred at position $q \geq r + 1$, then:
    - ★ set Z_k to $q - k$,
    - ★ set r to $q - 1$.
    - ★ set l to k.

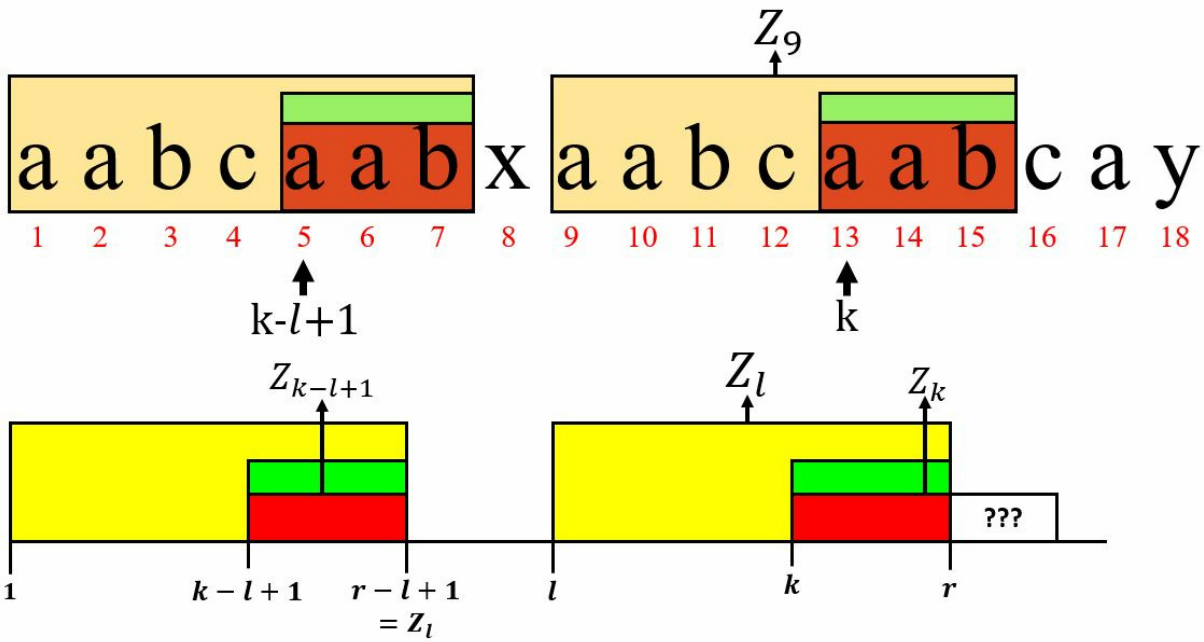


Z-algorithm – Case 2b mechanics



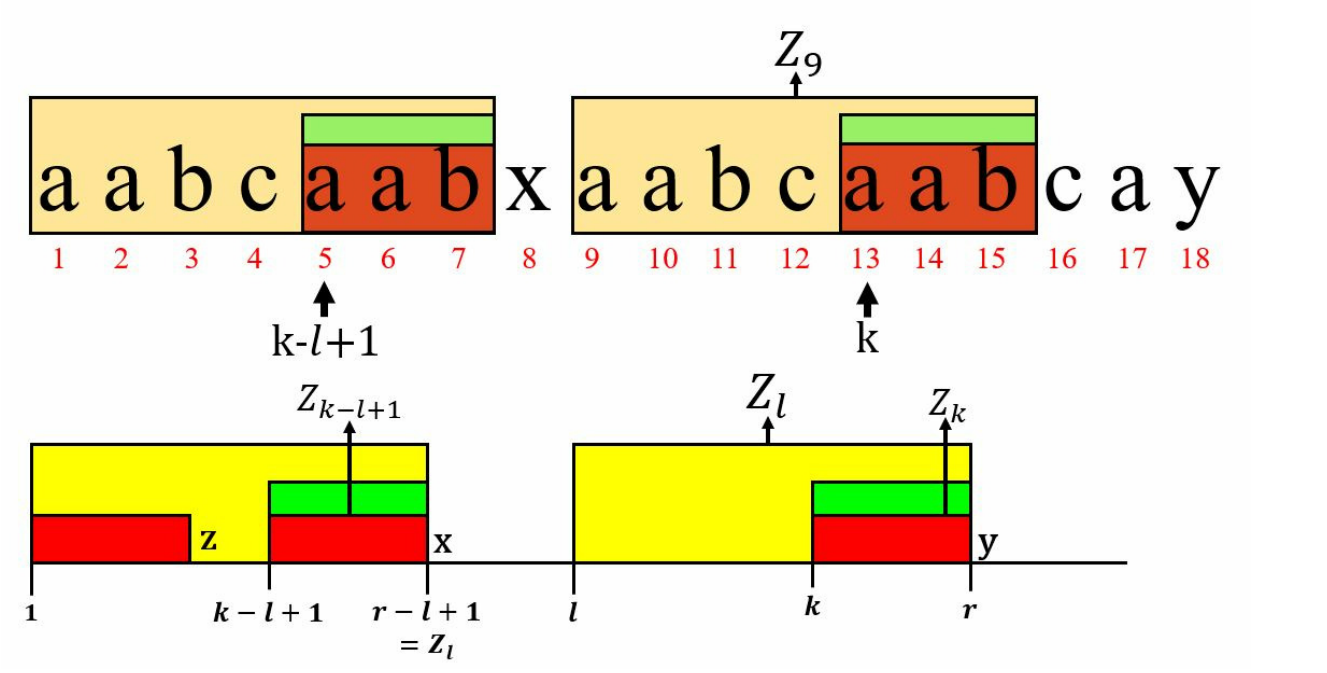
- By the definition of Z_l-box, **str**$[l \ldots r]$=**str**$[1 \ldots Z_l]$.
- This implies the character **str**$[k]$ is identical to **str**$[k-l+1]$
- By extension the substring **str**$[k \ldots r]$ is identical to **str**$[k-l+1 \ldots Z_l]$.

Z-algorithm – Case 2b mechanics



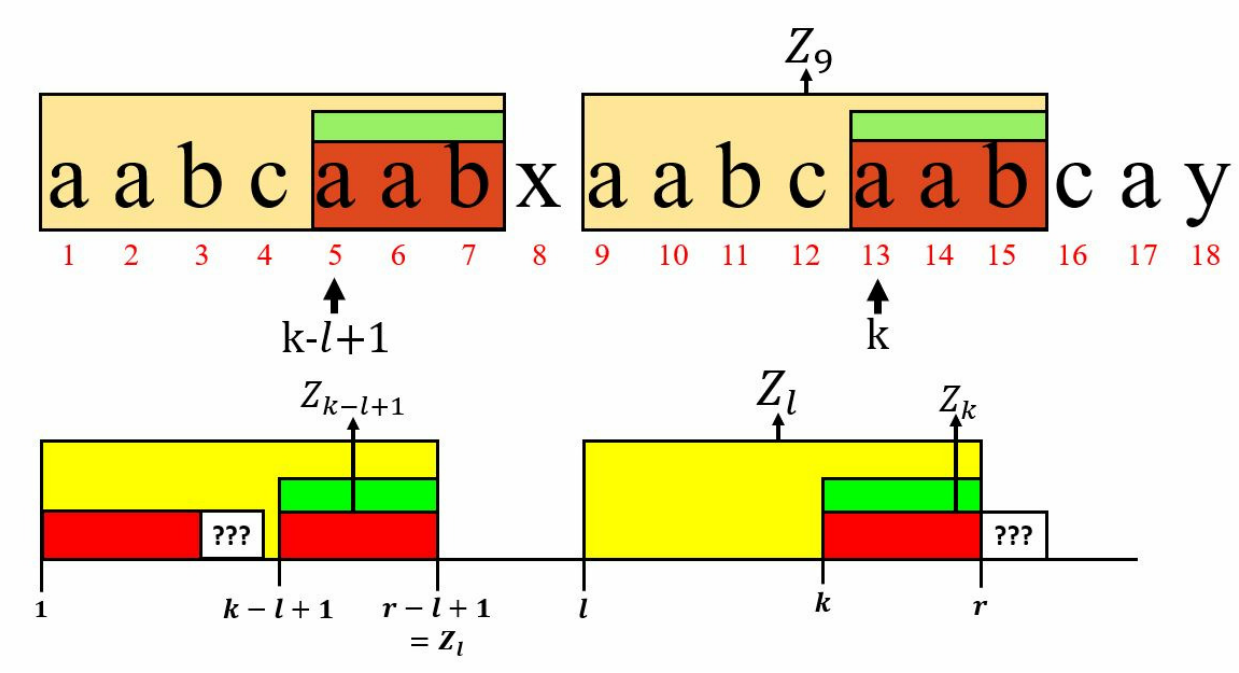
- By the definition of Z_l-box, **str**$[l \ldots r]$=**str**$[1 \ldots Z_l]$.
- This implies the character **str**$[k]$ is identical to **str**$[k-l+1]$
- By extension the substring **str**$[k \ldots r]$ is identical to **str**$[k-l+1 \ldots Z_l]$.
- But now $Z_{k-l+1} = r - k + 1$.

Z-algorithm – Case 2b mechanics



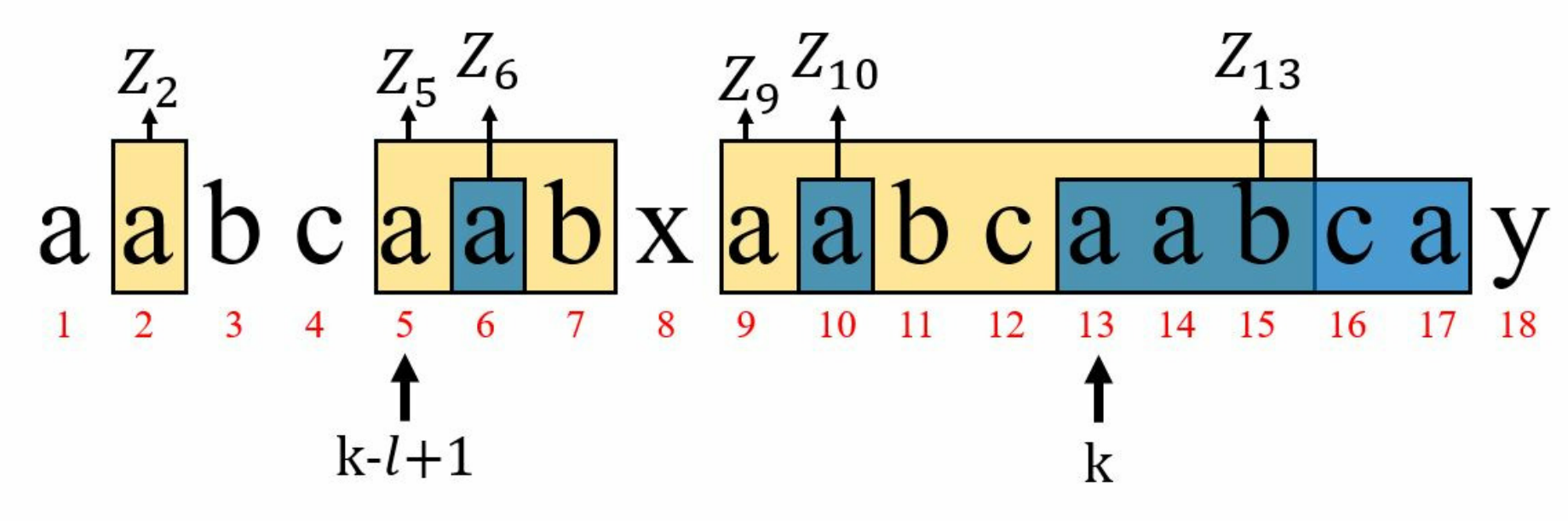
- By the definition of Z_l-box, **str**$[l \ldots r]$=**str**$[1 \ldots Z_l]$.
- This implies the character **str**$[k]$ is identical to **str**$[k-l+1]$
- By extension the substring **str**$[k \ldots r]$ is identical to **str**$[k-l+1 \ldots Z_l]$.
- But now $Z_{k-l+1} = r - k + 1$.
- What info have we gained so far? Ans: $x \neq y$, $z \neq x$. But we remain uncertain about the relationship between y and z.

Z-algorithm – Case 2b mechanics



- By the definition of Z_l-box, **str**$[l \ldots r]$=**str**$[1 \ldots Z_l]$.
- This implies the character **str**$[k]$ is identical to **str**$[k-l+1]$
- By extension the substring **str**$[k \ldots r]$ is identical to **str**$[k-l+1 \ldots Z_l]$.
- But now $Z_{k-l+1} = r - k + 1$.
- What info have we gained so far? Ans: $x \neq y$, $z \neq x$. But we remain uncertain about the relationship between y and z.
- Thus, explicit comprisons needed only in the regions of uncertainty.

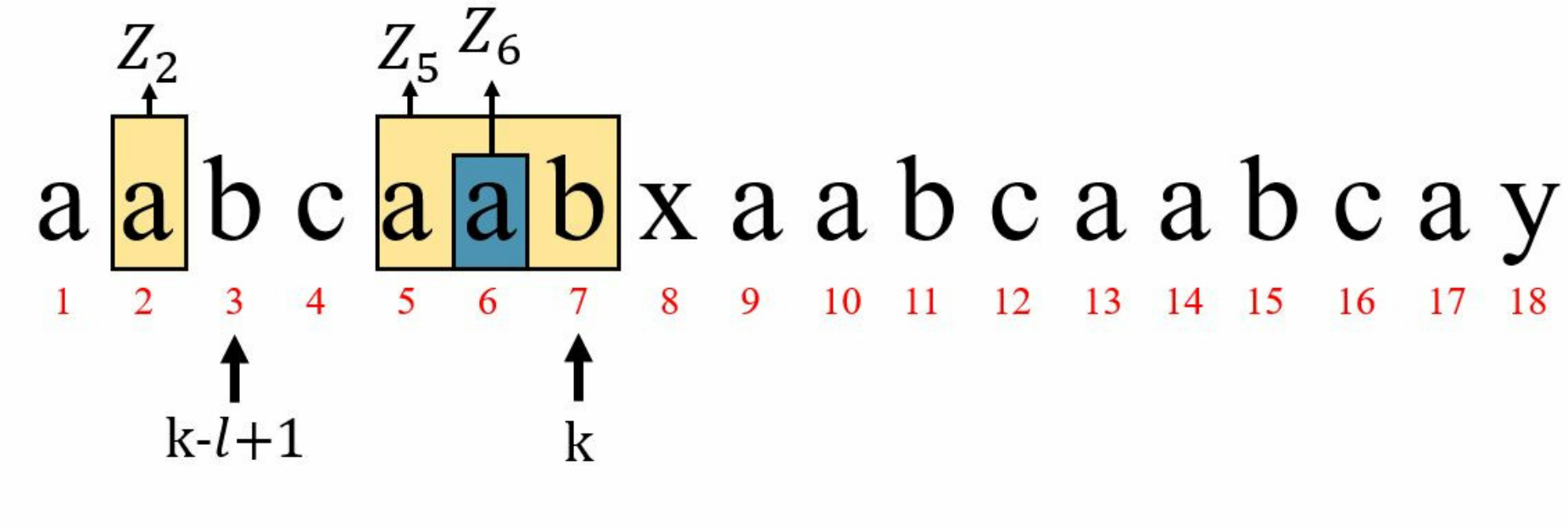
Z-algorithm – Case 2b Example



Perform explicit comparisons until mismatch between characters **str**$[16\ldots]$ and **str**$[4\ldots]$. Initially, $l = 9$ and $r = 15$, so $r + 1 = 16$ and $r - k + 2 = 4$. After computation of Z_{13}, the l and r values are updated to $l = 13$ and $r = 17$.

Z-algorithm: continuing iterations beyond those already covered – self-study

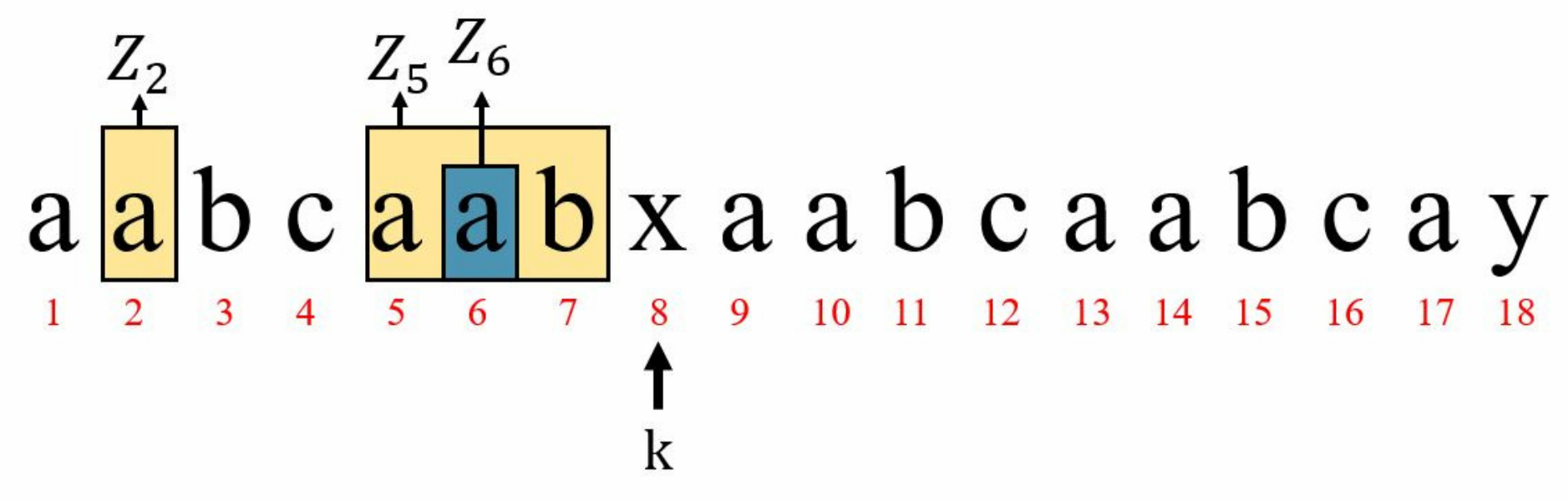
On the slide series #26 and #29, examples iterations $k = 2 \ldots 5$ and $k = 6$ were covered, respectively. Below we have continue the running example for $k \geq 7$.



Case 2a: $Z_7 = 0$ so l and r remain unchanged ($l = 5$ and $r = 7$).

Z-algorithm: continuing iterations beyond those already covered – self-study

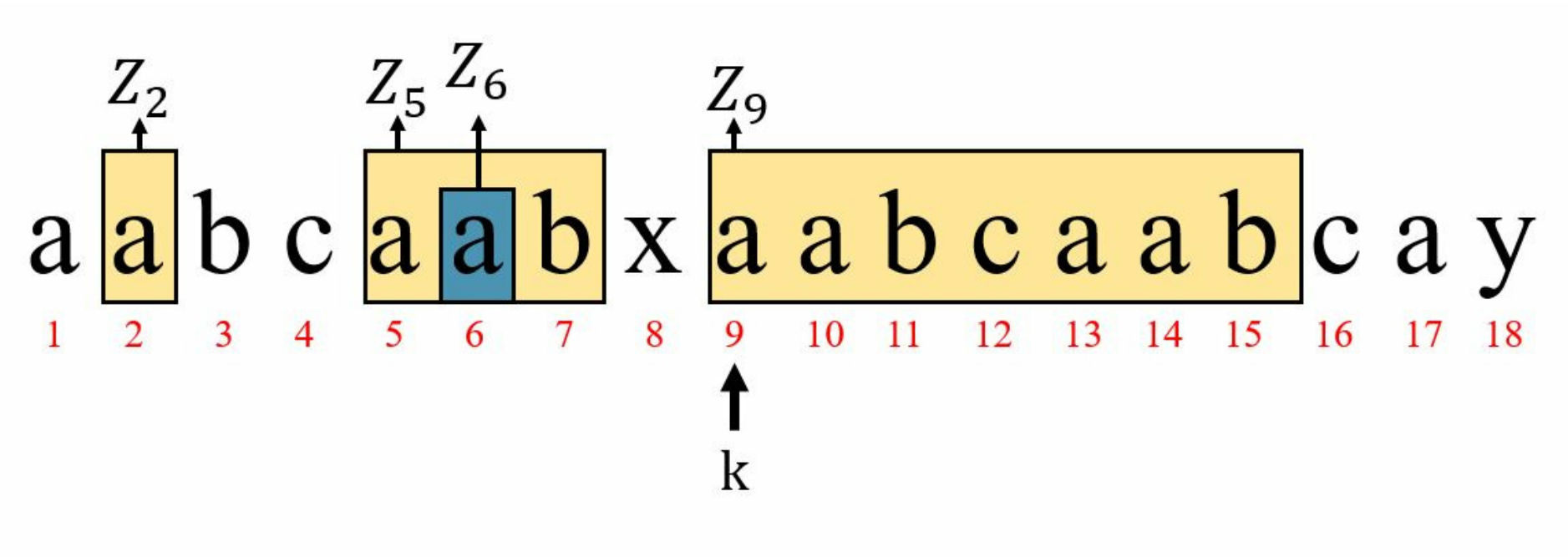
On the slide series #26 and #29, examples iterations $k = 2 \ldots 5$ and $k = 6$ were covered, respectively. Below we have continue the running example for $k \geq 7$.



Case 1: $Z_8 = 0$ so l and r remain unchanged ($l = 5$ and $r = 7$).

Z-algorithm: continuing iterations beyond those already covered – self-study

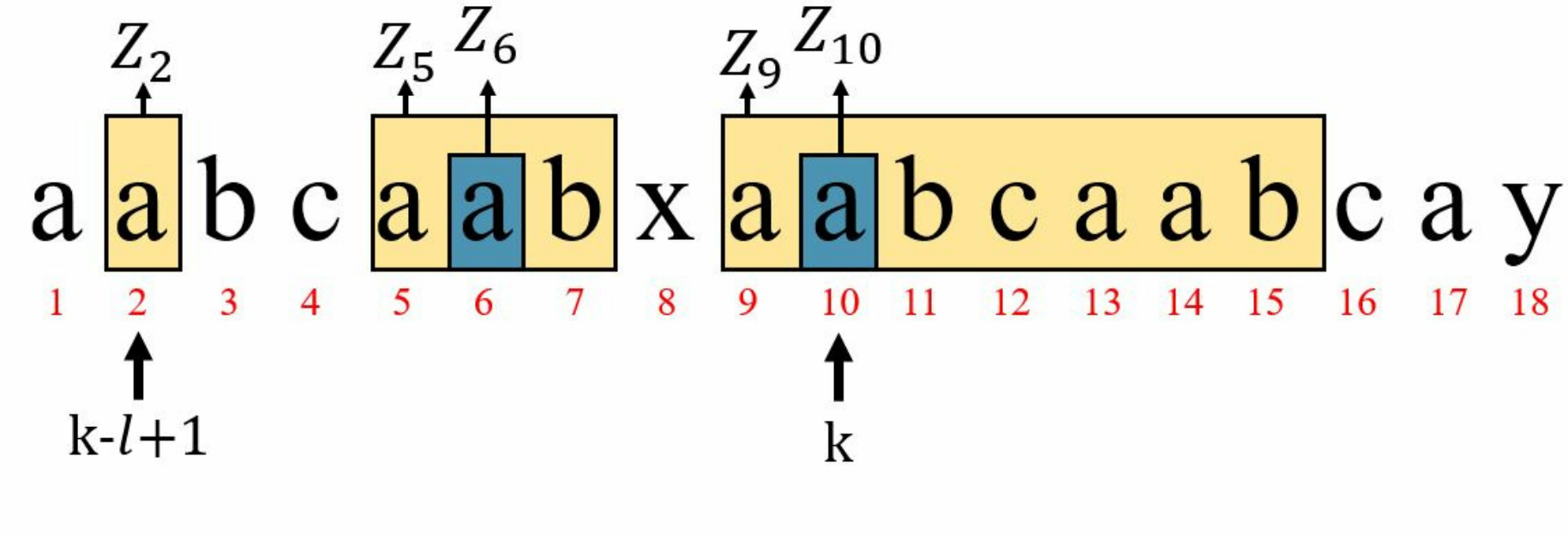
On the slide series #26 and #29, examples iterations $k = 2 \ldots 5$ and $k = 6$ were covered, respectively. Below we have continue the running example for $k \geq 7$.



Case 1: $Z_9 = 7$ so l and r are updated to: $l = 9$ and $r = 15$.

Z-algorithm: continuing iterations beyond those already covered – self-study

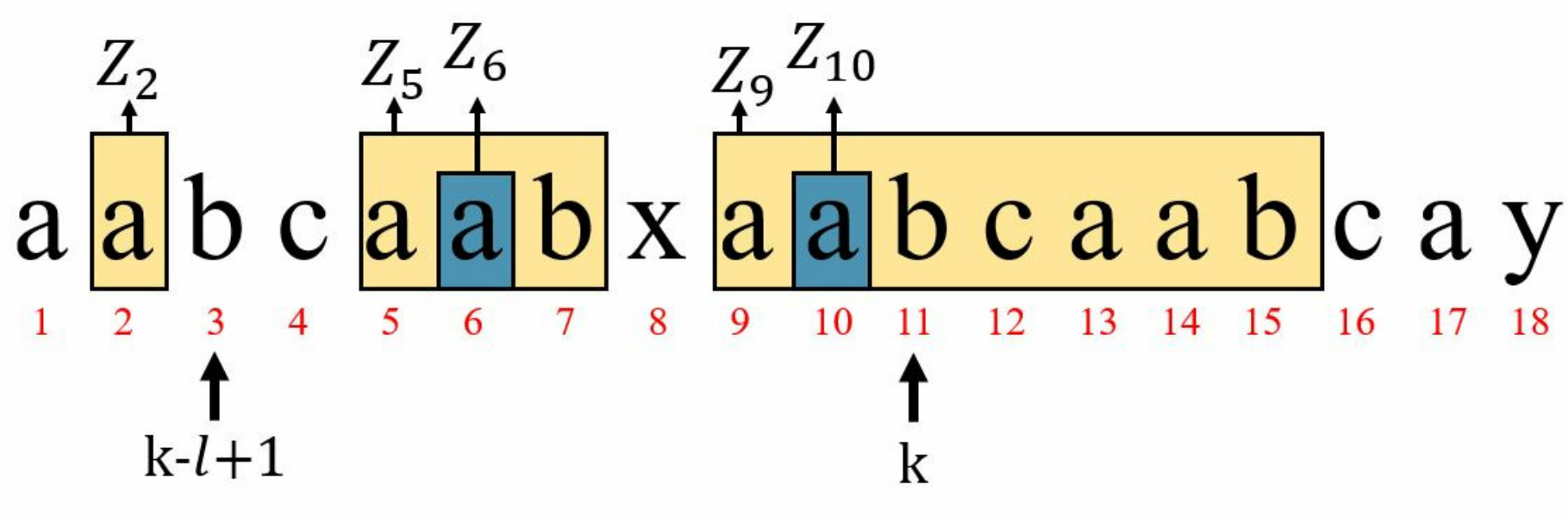
On the slide series #26 and #29, examples iterations $k = 2 \ldots 5$ and $k = 6$ were covered, respectively. Below we have continue the running example for $k \geq 7$.



Case 2a: $Z_{10} = 1$ so l and r remain unchanged ($l = 9$ and $r = 15$).

Z-algorithm: continuing iterations beyond those already covered – self-study

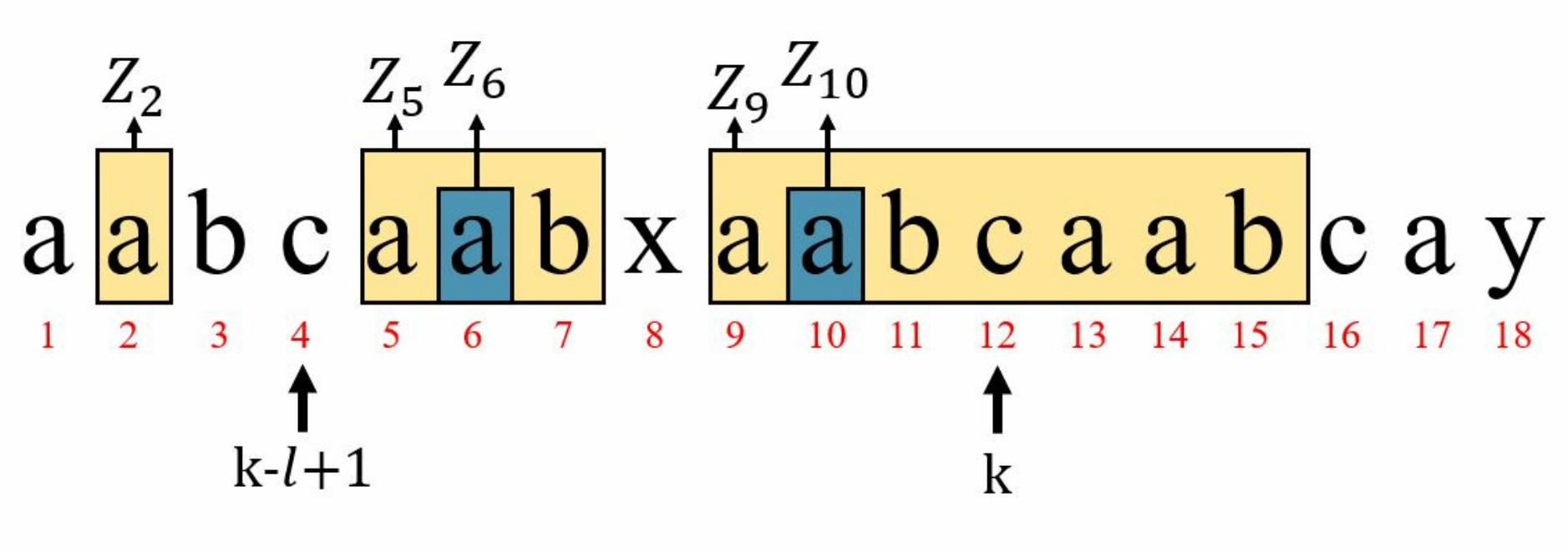
On the slide series #26 and #29, examples iterations $k = 2 \ldots 5$ and $k = 6$ were covered, respectively. Below we have continue the running example for $k \geq 7$.



Case 2a: $Z_{11} = 0$ so l and r remain unchanged ($l = 9$ and $r = 15$).

Z-algorithm: continuing iterations beyond those already covered – self-study

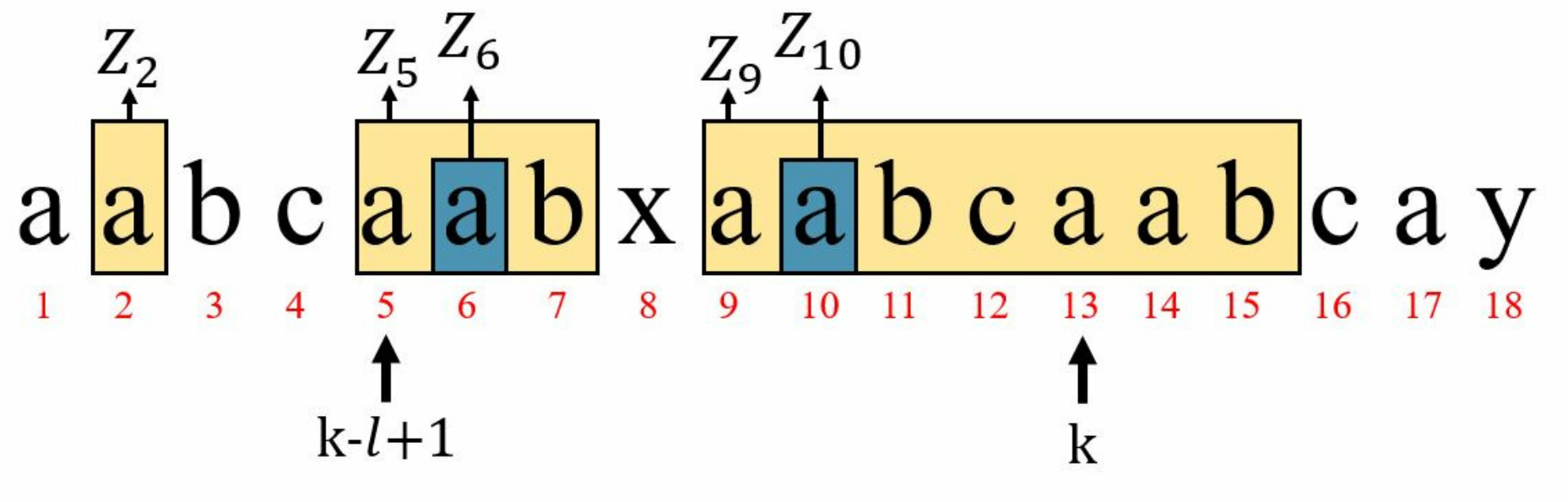
On the slide series #26 and #29, examples iterations $k = 2 \ldots 5$ and $k = 6$ were covered, respectively. Below we have continue the running example for $k \geq 7$.



Case 2a: $Z_{12} = 0$ so l and r remain unchanged ($l = 9$ and $r = 15$).

Z-algorithm: continuing iterations beyond those already covered – self-study

On the slide series #26 and #29, examples iterations $k = 2 \ldots 5$ and $k = 6$ were covered, respectively. Below we have continue the running example for $k \geq 7$.



Case 2b (see slide #32): $Z_{13} = 5$ (but needs only 3 extra comparisons, half of those required for the naive approach).

Preprocessing (of Z_i values) for **str**$[1..n]$ takes $O(n)$ time

- Total time is directly proportional to the SUM of:

Preprocessing (of Z_i values) for **str**$[1..n]$ takes $O(n)$ time

- Total time is directly proportional to the SUM of:
  1. the number of iterations

Preprocessing (of Z_i values) for **str**$[1..n]$ takes $O(n)$ time

- Total time is directly proportional to the SUM of:
  1. the number of iterations
  2. the number of explicit character comparisons (**matches** or **mismatches**).

Preprocessing (of Z_i values) for **str**$[1..n]$ takes $O(n)$ time

- Total time is directly proportional to the SUM of:
  1. the number of iterations
  2. the number of explicit character comparisons (**matches** or **mismatches**).
- This preprocessing has $n - 1$ iterations.

Preprocessing (of Z_i values) for **str**$[1..n]$ takes $O(n)$ time

- Total time is directly proportional to the SUM of:
  1. the number of iterations
  2. the number of explicit character comparisons (**matches** or **mismatches**).
- This preprocessing has $n - 1$ iterations.
- Any iteration that explicitly performs comparisons ends as soon as a mismatch occurs, hence there are at most $n - 1$ mismatches.

Preprocessing (of Z_i values) for **str**$[1..n]$ takes $O(n)$ time

- Total time is directly proportional to the SUM of:
  1. the number of iterations
  2. the number of explicit character comparisons (**matches** or **mismatches**).
- This preprocessing has $n-1$ iterations.
- Any iteration that explicitly performs comparisons ends as soon as a mismatch occurs, hence there are at most $n-1$ mismatches.
- The number of **matches** (perfomed explicitly) is at most n, because:

Preprocessing (of Z_i values) for **str**$[1..n]$ takes $O(n)$ time

- Total time is directly proportional to the SUM of:
  1. the number of iterations
  2. the number of explicit character comparisons (**matches** or **mismatches**).
- This preprocessing has $n-1$ iterations.
- Any iteration that explicitly performs comparisons ends as soon as a mismatch occurs, hence there are at most $n-1$ mismatches.
- The number of **matches** (perfomed explicitly) is at most n, because:
  - $r_k \geq r_{k-1}$ (for all iterations)

Preprocessing (of Z_i values) for **str**$[1..n]$ takes $O(n)$ time

- Total time is directly proportional to the SUM of:
  1. the number of iterations
  2. the number of explicit character comparisons (**matches** or **mismatches**).
- This preprocessing has $n-1$ iterations.
- Any iteration that explicitly performs comparisons ends as soon as a mismatch occurs, hence there are at most $n-1$ mismatches.
- The number of **matches** (perfomed explicitly) is at most n, because:
  - $r_k \geq r_{k-1}$ (for all iterations)
  - Update to r_k is of the form $r_k = r_{k-1} + \delta$ (where $\delta \geq 0$).

Preprocessing (of Z_i values) for **str**$[1..n]$ takes $O(n)$ time

- Total time is directly proportional to the SUM of:
  1. the number of iterations
  2. the number of explicit character comparisons (**matches** or **mismatches**).
- This preprocessing has $n-1$ iterations.
- Any iteration that explicitly performs comparisons ends as soon as a mismatch occurs, hence there are at most $n-1$ mismatches.
- The number of **matches** (perfomed explicitly) is at most n, because:
  - $r_k \geq r_{k-1}$ (for all iterations)
  - Update to r_k is of the form $r_k = r_{k-1} + \delta$ (where $\delta \geq 0$).
  - But $r_k \leq n$.

Preprocessing (of Z_i values) for **str**$[1..n]$ takes $O(n)$ time

- Total time is directly proportional to the SUM of:
  1. the number of iterations
  2. the number of explicit character comparisons (**matches** or **mismatches**).
- This preprocessing has $n-1$ iterations.
- Any iteration that explicitly performs comparisons ends as soon as a mismatch occurs, hence there are at most $n-1$ mismatches.
- The number of **matches** (perfomed explicitly) is at most n, because:
  - $r_k \geq r_{k-1}$ (for all iterations)
  - Update to r_k is of the form $r_k = r_{k-1} + \delta$ (where $\delta \geq 0$).
  - But $r_k \leq n$.
  - Thus, there are at most n **matches**.

Using Z-algorithm for linear-time exact pattern matching

Recall the exact pattern matching problem

Given a reference text **txt**$[1 \ldots n]$ and a pattern **pat**$[1 \ldots m]$, find **ALL** occurrences, if any, of **pat** in **txt**.

Using Z-algorithm for linear-time exact pattern matching

Recall the exact pattern matching problem

Given a reference text $\texttt{txt}[1 \ldots n]$ and a pattern $\texttt{pat}[1 \ldots m]$, find **ALL** occurrences, if any, of **pat** in **txt**.

Realizing a linear-time solution using Gusfield's Z-algorithm/preprocessing

- Construct a new string **str** by concatenation as follows:
  $\texttt{str} = \texttt{pat}[1 \ldots m] + \$ + \texttt{txt}[1 \ldots n]$.
  Note, $|\texttt{str}| = m + 1 + n$.

Using Z-algorithm for linear-time exact pattern matching

Recall the exact pattern matching problem

Given a reference text **txt**$[1 \ldots n]$ and a pattern **pat**$[1 \ldots m]$, find **ALL** occurrences, if any, of **pat** in **txt**.

Realizing a linear-time solution using Gusfield's Z-algorithm/preprocessing

- Construct a new string **str** by concatenation as follows:
  str $=$ **pat**$[1 \ldots m] + \$ +$ **txt**$[1 \ldots n]$.
  Note, $|$**str**$| = m + 1 + n$.
- Preprocess Z_i values corresponding to **str**, for $1 < i \leq m + n + 1$.

Using Z-algorithm for linear-time exact pattern matching

Recall the exact pattern matching problem

Given a reference text **txt**$[1 \ldots n]$ and a pattern **pat**$[1 \ldots m]$, find **ALL** occurrences, if any, of **pat** in **txt**.

Realizing a linear-time solution using Gusfield's Z-algorithm/preprocessing

- Construct a new string **str** by concatenation as follows:
  str $=$ **pat**$[1 \ldots m] + \$ +$ **txt**$[1 \ldots n]$.
  Note, $|$**str**$| = m + 1 + n$.
- Preprocess Z_i values corresponding to **str**, for $1 < i \leq m + n + 1$.
- For any $i > m + 1$, all $Z_i = m$ identifies an occurrence of **pat**[1 . . . m] at position i in **str**, and hence at position $i - (m + 1)$ in **txt**. That is, **pat**$[1 \ldots m] =$ (**str**$[i \ldots i + m - 1] \equiv$ **txt**$[i - (m + 1) \ldots i - 2]$).

Using Z-algorithm for linear-time exact pattern matching

Recall the exact pattern matching problem

Given a reference text **txt**$[1 \ldots n]$ and a pattern **pat**$[1 \ldots m]$, find **ALL** occurrences, if any, of **pat** in **txt**.

Realizing a linear-time solution using Gusfield's Z-algorithm/preprocessing

- Construct a new string **str** by concatenation as follows:
  str $=$ **pat**$[1 \ldots m] + \$ +$ **txt**$[1 \ldots n]$.
  Note, $|$**str**$| = m + 1 + n$.
- Preprocess Z_i values corresponding to **str**, for $1 < i \leq m + n + 1$.
- For any $i > m + 1$, all $Z_i = m$ identifies an occurrence of **pat**[1 . . . m] at position i in **str**, and hence at position $i - (m + 1)$ in **txt**. That is, **pat**$[1 \ldots m] =$ (**str**$[i \ldots i + m - 1] \equiv$ **txt**$[i - (m + 1) \ldots i - 2]$).
- We already, showed that the computation of Z_i values for any sting **str** takes $O(|$**str**$|)$ time.

Using Z-algorithm for linear-time exact pattern matching

Recall the exact pattern matching problem

Given a reference text **txt**$[1 \ldots n]$ and a pattern **pat**$[1 \ldots m]$, find **ALL** occurrences, if any, of **pat** in **txt**.

Realizing a linear-time solution using Gusfield's Z-algorithm/preprocessing

- Construct a new string **str** by concatenation as follows:
  str $=$ **pat**$[1 \ldots m] + \$ +$ **txt**$[1 \ldots n]$.
  Note, $|$**str**$| = m + 1 + n$.
- Preprocess Z_i values corresponding to **str**, for $1 < i \leq m + n + 1$.
- For any $i > m + 1$, all $Z_i = m$ identifies an occurrence of **pat**[1 . . . m] at position i in **str**, and hence at position $i - (m + 1)$ in **txt**. That is, **pat**$[1 \ldots m] =$ (**str**$[i \ldots i + m - 1] \equiv$ **txt**$[i - (m + 1) \ldots i - 2]$).
- We already, showed that the computation of Z_i values for any sting **str** takes $O(|$**str**$|)$ time.
- Thus, this pattern matching algorithm takes $O(m + n)$ time. **QED**

Lecture Summary

- We learnt Gusfield's Z-algorithm for string preprocessing.
- It has versatile uses, including pattern matching.
- Naïve pattern matching takes $O(m \times n)$-time.
- Z algorithm guarantees pattern matching in $O(n + m)$-time.

Next week ...

we will learn and analyze

- the famous Boyer-Moore's algorithm for pattern matching
  - which takes $O(n + m)$-time worst case ...
  - ... but $O(\frac{n}{m})$-time in most 'realworld' usage.
- the Knuth-Morris-Pratt algorithm also takes $O(n + m)$-time worst-case, but inferior in performance to Boyer-Moore in practice.

For both, we will make use of the Z-algorithm we learnt here. So, as pre-class learning for next week, do not forget to fully internalize the Z-algorithm.

-=o0o=-

END

-=o0o=-